1. Session Overview & Objectives

Welcome to Session 3! Today, we are taking your initial webpage and transforming it into a professional, adaptable interface. Building technology in a diverse community means creating products that work for everyone, regardless of what device they use or h ow they navigate the web. Remember, every expert was once a beginner, and refining your code step -by-step is a universal engineering reality, not a personal failure.

What We Will Learn:

- **Semantic HTML:** Using specific HTML tags that describe their content (like <article> or <nav>) rather than just acting as generic containers.
- **Accessibility (WCAG/POUR):** Designing web experiences that are usable by people with visual, motor, or cognitive disabilities.
- **Responsive Layouts (Flexbox):** Writing CSS that allows your website to automatically adjust and look great on a large desktop screen or a small mobile phone.
- **Responsible AI Integration:** Using tools like ChatGPT or v0 as collaborative assistants to improve your code, rather than blindly copying and pasting.

2. Conceptual Deep Dive

Building a modern website requires balancing design with structure and usability.

- **Semantic HTML:** Think of Semantic HTML like correctly labeling boxes when moving to a new house in Malé. If a box is labeled"Kitchen Supplies," anyone (or any computer) knows exactly what is inside without opening it. Tags like <header>, <main>, and <footer> tell web browsers and screen readers exactly how your page is organized.
- **Accessibility & WCAG/POUR:** The Web Content Accessibility Guidelines (WCAG) ensure digital inclusivity based on four principles (POUR):
  - **Perceivable:** Users must be able to see or hear the content. This means using **alt text** on images so screen readers can describe them, and ensuring high color contrast between text and backgrounds.
  - **Operable:** Users must be able to navigate the interface. If a user relies on a keyboard instead of a mouse, they should still be able to click links. For mobile touchscreens, buttons must be at least 24x24 CSS pixels so they are easy to tap.
  - **Understandable:** The layout and text should be predictable and easy to read.
  - **Robust:** The code must work across different devices and assistive technologies.
- **Responsive Layouts & Flexbox:** Flexbox is a CSS layout module that arranges elements dynamically. Imagine arranging seats on a dhoni; if more people board, the seating arrangement shifts to accommodate the space efficiently. Flexbox allows your webpage elements to shrink, grow, and wra p based on the screen size.

Knowledge Check: If an image fails to load on a weak internet connection, what specific HTML attribute ensures the user still knows what the image represents? Answer: alt (It puts the label to the image)

3. Tools & GitHub Workflow

We will continue using our professional development environment to build and store code.

Tool Setup:

1. Open **VS Code** or **Replit** and load the project repository you created in Session 2.
2. Open your preferred AI assistant in a browser tab (e.g., ChatGPT, v0, or GitHub Copilot).

The Git/GitHub Focus: Version control is an everyday professional habit. Before committing new changes today, always check what you are about to save.

1. Check your current file status: git status
2. Stage your layout changes: git add .
3. Commit with a meaningful message: git commit -m "Update responsive layout and accessibility"
4. Upload to GitHub: git push

AI Integration Note: Generative AI is a force multiplier for productivity, but it is not a copy-paste shortcut. You must use it as a collaborative assistant. The absolute rule of this program is that you must be able to independently explain any code you commit to your repository.

4. Practical Activity (Step-by-Step Walkthrough)

Today, you will prompt an AI tool to suggest an improved, accessible layout, compare it with your own code, and selectively integrate the improvements.

Step 1: Document Your AI Prompt

Open your AI tool and write a clear, descriptive prompt.

Example Prompt: "I have a basic HTML webpage with a heading, an introduction paragraph, and an image. Can you suggest a responsive CSS Flexbox layout for this content? Please ensure the colors have high contrast for accessibility and use semantic HTML tags."

Step 2: Review and Compare (Code Along)

Do not copy the entire AI response. Review the suggested CSS. Look for the display: flex; property. Notice how the AI might group your content inside a <main> tag instead of a generic .

Step 3: Selective Integration (Independent Practice)

1. Update your index.html file to replace generic tags with semantic tags (<header>, <main>, <section>).
2. Ensure every <img> tag in your HTML has descriptive alt="..." text.
3. In your style.css file, apply the Flexbox properties suggested by the AI to a container class (e.g., .profile-container).

Step 4: Debugging Tips If your layout looks broken, inspect your CSS for missing semicolons (;) or unclosed brackets (}). Treat error messages or visual bugs as helpful diagnostic tools, not personal failures.

5. Answer Key & Expected Output

Your updated files should resemble the following structure, demonstrating semantic HTML, accessibility, and Flexbox integration.

HTML (index.html):

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <!-- Ensures the page is responsive on mobile devices -->
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <link rel="stylesheet" href="style.css">
  <title>My Tech Profile</title>
</head>
<body>
  <!-- Semantic Header -->
  <header>
    <h1>Welcome to My Tech Journey</h1>
  </header>

  <!-- Semantic Main Content Area -->
  <main class="flex-container">
    <section class="text-content">
      <p>Hello! I am learning to build inclusive digital products in the Maldives.</p>
      <!-- Operable: Large, touch-friendly link -->
      <a href="https://github.com" class="action-button">View my GitHub</a>
    </section>

    <!-- Perceivable: Descriptive alt text included -->
    <img src="maldives.jpg" alt="A sunny view of a Maldivian island coastline" class="profile-
img">
  </main>
</body>
</html>
```

CSS (style.css):

```
/* High contrast colors for accessibility */
body {
  background-color: #ffffff;
  color: #1a1a1a;
  font-family: Arial, sans-serif;
  margin: 0;
  padding: 20px;
}

/* Responsive Flexbox Layout */
.flex-container {
```

```
  display: flex;
  flex-wrap: wrap; /* Allows items to stack on small screens */
  gap: 20px;
  align-items: center;
}

.text-content {
  flex: 1; /* Takes up available space */
  min-width: 250px;
}

.profile-img {
  max-width: 100%;
  height: auto;
  border-radius: 8px;
}

/* Accessible touch target (min 24x24px) */
.action-button {
  display: inline-block;
  padding: 12px 24px;
  background-color: #0056b3; /* High contrast blue */
  color: white;
  text-decoration: none;
  border-radius: 5px;
}
```

Visual Output: On a wide desktop screen, the text and image will sit side-by-side. If you shrink the browser window to simulate a mobile phone, the image will automatically drop below the text (thanks to flex-wrap: wrap).

Common Mistakes:

- **Missing alt text:** Leaving alt="" empty. Always describe the image for screen readers.
- **Low Contrast:** Using light gray text on a white background. Ensure text is dark enough to read easily.
- **Overwriting Code Blindly:** Pasting the AI's entire HTML file over your own, accidentally deleting your personal text or image links.

6. Deliverables Checklist

Before concluding today's session, ensure you have completed the following:

- Responsive, accessible webpage completed using Flexbox and semantic HTML.
- Documented AI prompt saved in your repository or notes.
- Changes successfully staged, committed, and pushed to your GitHub repository.
- Checked your repository online at GitHub.com to verify the new commit history is visible.